

Visión artificial

Sesión 4.Homografías asociadas y calibración

Matemática Aplicada

Dept. Ingeniería y Tecnología de Computadores

Universidad de Murcia

Máster en Inteligencia Artificial

2026/2027



Repaso

Homografías inducidas por la matriz de cámara M

Otros métodos de calibración

El método de Zhang

Calibración por puntos de fuga

Recuperando la pose

Repaso



Tanto al estimar una homografía H (3×3) como una matriz de cámara M (3×4), partimos de una relación de igualdad salvo escala:

$$x_i \sim PX_i \implies \lambda x_i = PX_i$$

Para eliminar el factor de escala desconocido λ , aplicamos el producto vectorial. Como un vector es paralelo a sí mismo, su producto vectorial es nulo:

$$x_i \times (PX_i) = 0$$

Sistema de ecuaciones: Cada correspondencia tridimensional i nos aporta 2 ecuaciones lineales. Agrupamos las incógnitas en una columna h :

$$Ah = 0$$



Herramientas para "desmontar" matrices

1. Descomposición en Valores Singulares (SVD): $A = U\Sigma V^T$

- ▶ Divide cualquier matriz A en dos matrices ortogonales (U, V) y una diagonal (Σ) con los valores singulares.
- ▶ *Uso principal*: Encontrar el espacio nulo de un sistema homogéneo $Ah = 0$.

2. Factorización QR / RQ:

- ▶ Descompone una matriz en el producto de una ortogonal (Q o R_{ort}) y una triangular (R_{triang} o K).
- ▶ *Uso principal (RQ)*: Dada $M = [M_{3 \times 3} | m_4]$, aplicamos RQ. La factorización nos separa la matriz intrínseca K de la orientación física R .



El problema de la inestabilidad numérica

Al construir la matriz A del DLT puede ser que SVD falle en la precisión.

1. Normalización (Richard Hartley): Antes de plantear el DLT, transformamos los puntos x_i y X_i aplicando una traslación y un escalado previo

2. Resolución del Sistema:

- ▶ Construimos A_{norm} con los puntos normalizados.
- ▶ Aplicamos SVD a A_{norm} .
- ▶ *Desnormalización:* Reorganizamos el vector h en matriz y multiplicamos por las inversas de las matrices de normalización para recuperar la matriz H o M real.

¿Qué sabemos hacer hasta ahora?



1. Transformaciones Planas (Geometría 2D)

- ▶ **Construcción de Homografías:** Estimación de la matriz H a partir de 4 correspondencias manuales (DLT 2D).
- ▶ **Rectificación de perspectiva:** "Enderezar" fotográficamente un plano oblicuo.
- ▶ **Metrología plana:** Medir distancias físicas en el suelo o paredes usando patrones de referencia.
- ▶ **Inserción de elementos:** Deformar texturas virtuales para anclarlas en la perspectiva de una foto real.

¿Qué sabemos hacer hasta ahora?



2. Modelo de Cámara 3D

- ▶ **Calibración básica:** Ya sabemos calcular la matriz de cámara $M = K[R|t]$ (mediante DLT 3D) usando un objeto físico tridimensional conocido.
- ▶ **Extracción de parámetros:** Separar las propiedades de la lente (K) de la pose física de la cámara (R, t) usando la factorización RQ.

¿Qué queremos hacer ahora?



El objetivo de la Sesión 4

Hasta hoy, tratábamos H y M como herramientas separadas que dependían de marcar puntos a mano. Hoy veremos cómo interactúan matemáticamente entre sí y cómo automatizar su cálculo.

Homografías inducidas por la matriz de cámara M

¿Cuándo la proyección 3D se convierte en homografía 2D?



La matriz de cámara M proyecta puntos del espacio sobre el plano proyectivo de la imagen. Es una pérdida irreversible de dimensión.

- ▶ **La excepción bidimensional:** Existen dos escenarios físicos y geométricos donde esta transformación de proyección se reduce a una **homografía plana H**
- ▶ **Los dos casos naturales:**
 1. **Escena plana (marcadores):** Cuando todos los puntos físicos de interés de la escena se sitúan sobre un único plano geométrico real.
 2. **Rotación pura de la cámara:** Cuando capturamos varias imágenes variando la orientación del dispositivo pero **manteniendo fijo su centro óptico en el espacio**.

Caso 1: Homografía inducida por un plano



Definamos el sistema de coordenadas de la escena de modo que el plano de interés coincida con el plano $Z_w = 0$ del mundo real.

► **Desarrollo algebraico de la proyección:**

$$\mathbf{x} \sim K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ 0 \\ 1 \end{bmatrix} \quad (1)$$

► **Desaparición del tercer eje:** La columna de rotación $\mathbf{r}_3$ se multiplica por cero y se desvanece de la proyección:

$$\mathbf{x} \sim K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ 1 \end{bmatrix} \implies \mathbf{x} \sim H_{\text{inducida}} \mathbf{x}_{\text{planar}} \quad (2)$$

► **Resultado:** La matriz resultante es $H = K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix}$.



En las primeras clases ya explotamos con éxito este **Caso 1**:

- ▶ **Rectificación de perspectiva:** "Enderezar" un plano oblicuo.
- ▶ **Metrología plana:** Medir distancias en el suelo.

En aquel momento, estimábamos la matriz H directamente a partir de 4 correspondencias manuales (DLT), ignorando por completo la física de la lente o la posición de la cámara.

Ventaja actual:

Tras la Sesión 3, **ya sabemos calibrar la cámara** ($M = K[R | t]$).

Rectificar perspectivas o medir milímetros reales ahora es más directo y robusto y no dependemos de colocar plantillas físicas conocidas en la escena ni buscar correspondencias manuales en cada nueva foto.

Caso 2: Rotación pura



Para que dos imágenes del espacio 3D general se relacionen de forma exacta mediante una homografía plana sin importar la profundidad de la escena, **debemos evitar estrictamente el paralaje de movimiento.**

- ▶ **Paralaje:** Si la cámara se desplaza en el espacio ($t \neq 0$), los puntos cercanos se mueven más rápido en el sensor que los lejanos. La profundidad altera la proyección y rompe la homografía.
- ▶ **Rotación pura:** La cámara gira sobre su propio centro de proyección común sin desplazarse por lo que no hay paralaje. Los puntos a diferentes distancias alineados en un rayo visual seguirán alineados tras el giro.



Situemos el origen de coordenadas en el centro óptico común de dos cámaras con la misma matriz K . Sus matrices de proyección son $M_1 = K[R_1 \mid \mathbf{0}]$ y $M_2 = K[R_2 \mid \mathbf{0}]$, donde R_1 y R_2 representan las rotaciones en ambas tomas.

1. Proyección en la primera cámara: $\mathbf{x}_1 \sim KR_1\mathbf{X} \implies \mathbf{X} \sim R_1K^{-1}\mathbf{x}_1$
2. Proyección en la segunda cámara (bajo rotación R): $\mathbf{x}_2 \sim KR\mathbf{X}$
3. Homografía inducida por la rotación: $\mathbf{x}_2 \sim K(R_2R_1^{-1})K^{-1}\mathbf{x}_1$

Matriz de la homografía:

$$H_{\text{rot}} = KR_2R_1^{-1}K^{-1} \quad (3)$$

Nota: también se puede calcular la homografía si las dos cámaras no tienen la misma matriz K



¿Para qué usamos la relación matemática entre H y M ?

1. Enfoque Directo (Física $\rightarrow$ Homografía)

- ▶ Construimos H analíticamente usando parámetros conocidos ($K, R, t, Z = 0$).
- ▶ *Aplicaciones:*
 - ▶ **Generación de datos para IA:** Crear miles de perspectivas sintéticas de un objeto plano deformando su textura.
 - ▶ **Stitching con hardware:** Leer el giroscopio del móvil para obtener R_{rel} y ensamblar una panorámica al instante ($H = KR_{rel}K^{-1}$), sin el coste computacional de buscar píxeles.

2. Enfoque Inverso (Imagen $\rightarrow$ Homografía $\rightarrow$ Física)

- ▶ Calculamos H empíricamente buscando correspondencias de píxeles y luego la *desmontamos* con álgebra para deducir propiedades 3D.
- ▶ *Aplicaciones:*



La homografía $H = KRK^{-1}$ puede usarse para que los dispositivos móviles y programas de edición puedan coser fotografías del entorno en un mosaico común.

- ▶ **Concepto de lienzo común:** Podemos tomar múltiples imágenes girando sobre un trípode (centro común) y proyectar todas las tomas periféricas hacia el plano de una imagen de referencia mediante la estimación de sus homografías de transferencia.
- ▶ **Independencia de la escena:** Como la relación $\mathbf{x}_2 \sim H_{\text{rot}}\mathbf{x}_1$ no depende de las coordenadas Z de la escena, el fondo lejano y los objetos cercanos se cosen de forma matemáticamente perfecta, sin sufrir las dobles siluetas ni fantasmas provocados por el paralaje.

Image Stitching



Otros métodos de calibración



El límite del algoritmo DLT 3D

En la Sesión 3 vimos que el algoritmo DLT 3D requiere un objeto de calibración con geometría tridimensional conocida con precisión milimétrica (ej. cubo de calibración). Esto es caro de fabricar y difícil de mantener rígido.

- ▶ **El algoritmo de Zhang (1999):** Permite calibrar con un simple patrón plano fotografiado desde múltiples orientaciones libres
 - ▶ No requiere conocer la posición 3D de la cámara ni del patrón.
 - ▶ Es robusto, barato y se ha convertido en el **estándar de OpenCV**.
- ▶ **Calibración por puntos de fuga:** Una alternativa clásica consiste en usar la geometría de la escena (ej. buscar líneas paralelas que convergen en puntos de fuga ortogonales). Permite estimar la focal, pero es sensible a la detección manual de líneas.

El método de Zhang



- ▶ El método de Zhang le da la vuelta al razonamiento del Caso 1
- ▶ Hay casos en que pueden calcularse homografías sin pasar antes por M
- ▶ Un caso interesante es cuando los puntos están en un plano
- ▶ Podemos calcular varias de esas homografías y, a partir de ellas, reconstruir M o una parte de ella



- ▶ Situemos el plano del patrón de calibración en $Z_w = 0$. La proyección perspectiva de la cámara $M = K[R \mid \mathbf{t}]$ se reduce a una homografía plana $H = K [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{t}]$
- ▶ Sea $H = [\mathbf{h}_1 \quad \mathbf{h}_2 \quad \mathbf{h}_3]$, tendremos:

$$[\mathbf{h}_1 \quad \mathbf{h}_2 \quad \mathbf{h}_3] = \lambda K [\mathbf{r}_1 \quad \mathbf{r}_2 \quad \mathbf{t}] \quad (4)$$

- ▶ Esto permite "despejar" las columnas de rotación físicas:

$$\mathbf{r}_1 = \frac{1}{\lambda} K^{-1} \mathbf{h}_1, \quad \mathbf{r}_2 = \frac{1}{\lambda} K^{-1} \mathbf{h}_2 \quad (5)$$

Pero $\mathbf{r}_1$ y $\mathbf{r}_2$ son ortonormales luego:

1. Condición de ortogonalidad:

$$\mathbf{h}_1^T \left(K^{-T} K^{-1} \right) \mathbf{h}_2 = 0 \quad (6)$$

2. Condición de igualdad de norma:

$$\mathbf{h}_1^T \left(K^{-T} K^{-1} \right) \mathbf{h}_1 = \mathbf{h}_2^T \left(K^{-T} K^{-1} \right) \mathbf{h}_2 \quad (7)$$

La matriz IAC (ω)

Definimos la matriz simétrica y definida positiva $\omega = K^{-T} K^{-1}$. Las restricciones de Zhang quedan como:

$$\mathbf{h}_1^T \omega \mathbf{h}_2 = 0 \quad \text{y} \quad \mathbf{h}_1^T \omega \mathbf{h}_1 - \mathbf{h}_2^T \omega \mathbf{h}_2 = 0 \quad (8)$$



- ▶ ω es simétrica por lo que solo tiene 6 parámetros independientes:

$$\mathbf{b} = [\omega_{11}, \omega_{12}, \omega_{22}, \omega_{13}, \omega_{23}, \omega_{33}]^T \quad (9)$$

- ▶ Cada homografía H_j estimada a partir de una foto del tablero nos aporta **2 ecuaciones lineales** sobre los coeficientes de $\mathbf{b}$
- ▶ **¿Cuántas imágenes necesitamos?**
 - ▶ ω tiene 5 grados de libertad (matriz homogénea de escala indeterminada).
 - ▶ Cada imagen da 2 ecuaciones. Con **3 imágenes** obtenemos 6 ecuaciones, suficiente para resolver el sistema homogéneo:

$$V\mathbf{b} = \mathbf{0} \quad (10)$$

usando la **SVD** de la matriz V del sistema dado por todas las restricciones lineales.

¿Qué es la descomposición de Cholesky?



- ▶ Pero tenemos un problema, en vez de K tenemos ecuaciones para calcular $\omega = K^{-T} \cdot K^{-1}$
- ▶ La descomposición de Cholesky es una descomposición matemática aplicable a matrices reales, simétricas y definidas positivas. Permite factorizar una matriz A en el producto de una matriz triangular inferior L con valores positivos en su diagonal y su transpuesta L^T , $A = LL^T$
- ▶ Esta descomposición es única y actúa como una "raíz cuadrada" matricial.
- ▶ Es decir, en nuestro caso permite recuperar K^{-1} a partir de ω
- ▶ Es computacionalmente el doble de rápida y mucho más estable que la descomposición LU de propósito general



Pasos a seguir para recuperar la matriz K .

- ▶ Calcular ω ,
- ▶ Calcular su descomposición de Cholesky $\omega = LL^T$
- ▶ $K = L^{-T}$
- ▶ Como ω se estimó salvo escala homogénea, dividimos K por su término inferior derecho K_{33} para asegurar la restricción física $K_{33} = 1.0$.



Las librerías como OpenCV implementan el algoritmo de Zhang estructurado en tres grandes fases para asegurar la máxima robustez física:

1. **Fase lineal inicial (SVD + Cholesky):** Se detectan las esquinas de los tableros, se calculan las homografías H_j , se monta el sistema $V\mathbf{b} = \mathbf{0}$ para hallar ω y se extrae $K_{inicial}$ mediante Cholesky. Las extrínsecas iniciales se obtienen como $R_j = [\mathbf{r}_{1j} \ \mathbf{r}_{2j} \ \mathbf{r}_{1j} \times \mathbf{r}_{2j}]$.
2. **Modelado de distorsión de lente:** Se asumen coeficientes iniciales nulos para la distorsión radial (k_1, k_2) introducida por la curvatura del silicio y el vidrio de las lentes.
3. **Refinamiento no lineal (Bundle adjustment):** Se optimizan simultáneamente todos los parámetros intrínsecos (K, k_1, k_2) y extrínsecos ($R_j, \mathbf{t}_j$) mediante mínimos cuadrados no lineales (Levenberg-Marquardt), minimizando el error de reproyección:

$$\min_{K, k_i, R_j, \mathbf{t}_j} \sum_j \sum_i \|\mathbf{x}_{ji} - \hat{\mathbf{x}}(K, k_i, R_j, \mathbf{t}_j, \mathbf{X}_i)\|^2 \quad (11)$$

Calibración por puntos de fuga



las coordenadas homogéneas permiten tratar elementos ideales de forma unificada:

- ▶ **Punto en el infinito:** tiene la forma $\mathbf{X}_\infty = (X, Y, Z, 0)^T$
- ▶ **Recta/Plano del infinito:** La recta del infinito tiene la forma $\mathbf{l}_\infty = (0, 0, 1)^T$ y es el lugar geométrico donde se cortan todas las líneas paralelas del plano.
- ▶ **¿Cómo se proyecta un punto del infinito? (punto de fuga):** Al aplicar la proyección de cámara $M = K[R \mid \mathbf{t}]$ a un punto ideal, la traslación se anula, proyectándose como un punto finito:

$$\mathbf{v} \sim K\mathbf{R}\mathbf{d} \quad (12)$$

La proyección de este punto ideal es lo que físicamente llamamos **punto de fuga**.



Identificamos en el mundo real dos conjuntos de líneas paralelas que son estrictamente **ortogonales**, sus vectores de dirección cumplen:

$$\mathbf{d}_1^T \mathbf{d}_2 = 0 \quad (13)$$

1. Podemos calcular los puntos de fuga asociados $\mathbf{v}_1$ y $\mathbf{v}_2$

$$\mathbf{v}_i \sim KR\mathbf{d}_i \implies \mathbf{d}_i \sim R^T K^{-1} \mathbf{v}_i \quad (14)$$

2. La ecuación fundamental de calibración: Usando $R^T R = I$:

$$\mathbf{d}_1^T \mathbf{d}_2 = 0 \implies \mathbf{v}_1^T \omega \mathbf{v}_2 = 0 \quad (15)$$

Cada pareja de puntos de fuga ortogonales aporta una restricción lineal directa sobre $\omega = K^{-T} K^{-1}$.

Supongamos una cámara con píxel cuadrado, punto principal centrado en el origen $(0, 0)$ y sesgo nulo: $K = \text{diag}(f, f, 1)$.

- **Planteamiento:** Identificamos la línea de horizonte del suelo (l_∞) y dos puntos de fuga mutuamente perpendiculares sobre ella.
- **Coordenadas de la escena:** Situemos el horizonte a una distancia perpendicular h del centro óptico de la imagen. Las posiciones de los puntos de fuga en la imagen serán $\mathbf{v}_1 = (-x_1, h, 1)^T$ y $\mathbf{v}_2 = (x_2, h, 1)^T$.
- **Resolución de la Focal:** Aplicando la ortogonalidad espacial $\mathbf{d}_1^T \mathbf{d}_2 = 0$:

$$f = \sqrt{x_1 x_2 - h^2} \quad (16)$$

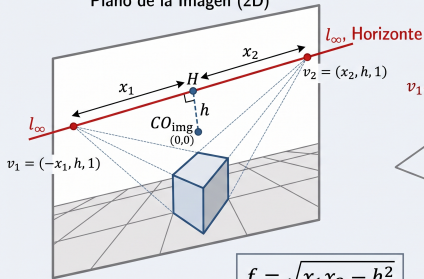


Pasos a seguir:

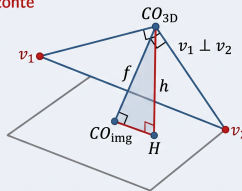
- ▶ Identificar los dos pares de paralelas $\{l_1, l_2\}$ y $\{l_3, l_4\}$ que sean perpendiculares entre sí
- ▶ Calcular el punto de fuga de cada par, v_i
- ▶ Usar esos dos puntos para calcular la línea del infinito
- ▶ Calcular la perpendicular del centro óptico, CO, de la imagen a esta línea y su corte, H, con esta línea
- ▶ Calcular $d = d(CO, H)$ y $x_i = d(v_i, H)$ para $i = 1, 2$
- ▶ Calcular $f = \sqrt{x_1 x_2 - d^2}$

Cálculo de la Focal (f) con Puntos de Fuga (Horizonte Inclinado)

Plano de la Imagen (2D)

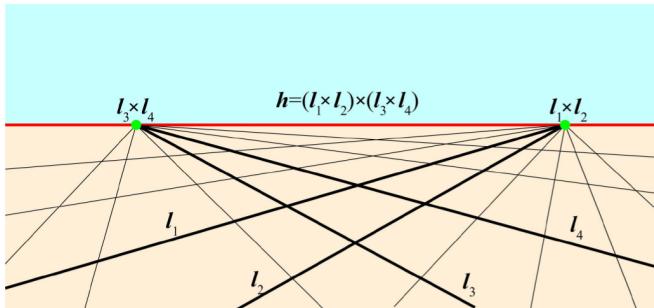


Relación Espacial (3D)



$$f = \sqrt{x_1 x_2 - h^2}$$

Observación: La mayor parte de estos cálculos pueden simplificarse usando productos vectoriales



Recuperando la pose



¿Qué diferencia hay entre la calibración clásica y la AR?

- ▶ **Calibración (offline):** Usamos geometría conocida (ej. tablero de Zhang) para descubrir la matriz intrínseca K de nuestro dispositivo.
- ▶ **Rastreo / Pose tracking (tiempo real):** Asumimos que K ya está guardada en memoria. El objetivo ahora es hallar continuamente la orientación y posición de la cámara (la matriz de extrínsecos $[R|t]$) respecto al mundo real.
- ▶ Una vez calculada $[R|t]$, el motor gráfico (Unity, Unreal) puede inyectar un holograma 3D exactamente en el mismo sistema de coordenadas, logrando la ilusión óptica de la Realidad Aumentada.



El Marcador Plano

- ▶ En la AR clásica (ArUco, Vuforia), colocamos un marcador físico sobre una mesa.
- ▶ Definimos el origen del mundo 3D en el centro de ese marcador, con el plano de la mesa coincidiendo con $Z_w = 0$.

Como vimos previamente (Caso 1), la proyección de cualquier punto de la mesa a la pantalla se reduce a una homografía plana:

$$x \sim K \begin{bmatrix} r_1 & r_2 & t \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ 1 \end{bmatrix} \implies x \sim H_{\text{marcador}} X_{\text{planar}}$$



El sistema de visión artificial detecta las 4 esquinas del marcador en la imagen y calcula la matriz de homografía empírica H .

Sabemos que teóricamente H encierra los extrínsecos:

$$H = \begin{bmatrix} h_1 & h_2 & h_3 \end{bmatrix} = \lambda K \begin{bmatrix} r_1 & r_2 & t \end{bmatrix}$$

Como K es conocida, multiplicamos por su inversa para "limpiar" la proyección y despejar las columnas físicas:

$$\begin{bmatrix} r_1 & r_2 & t \end{bmatrix} = \frac{1}{\lambda} K^{-1} \begin{bmatrix} h_1 & h_2 & h_3 \end{bmatrix}$$



Recuperando el factor de escala λ

Tenemos las ecuaciones:

$$r_1 = \frac{1}{\lambda} K^{-1} h_1 \quad r_2 = \frac{1}{\lambda} K^{-1} h_2 \quad t = \frac{1}{\lambda} K^{-1} h_3$$

¿Cómo calculamos el valor de λ ?

- ▶ La matriz de rotación física R pertenece al grupo especial ortogonal $SO(3)$.
- ▶ Por tanto, sus columnas r_1 y r_2 deben ser vectores unitarios (norma euclídea igual a 1).

Imponiendo esta restricción física, el factor de escala homogéneo se revela directamente:

$$\lambda = \|K^{-1} h_1\| = \|K^{-1} h_2\|$$



Con λ calculado, ya tenemos r_1 (dirección del eje X del mundo) y r_2 (dirección del eje Y del mundo), además del vector de traslación t .

Falta el eje de profundidad (r_3). Al usar un marcador plano ($Z_w = 0$), la homografía "destruyó" la información del eje Z.

Solución geométrica: Dado que los tres ejes de un sistema de coordenadas cartesiano tridimensional son mutuamente perpendiculares y siguen la regla de la mano derecha, el tercer vector emerge mediante un producto vectorial puro:

$$r_3 = r_1 \times r_2$$



Si ensamblamos la matriz de rotación directamente $R_{sucia} = [r_1 \ r_2 \ r_3]$, nos enfrentamos a un problema crítico en la práctica:

- ▶ El cálculo de la homografía original H contiene ruido (resolución de la cámara, desenfoque, detección imprecisa de las esquinas).
- ▶ Por culpa del ruido, los vectores estimados r_1 y r_2 casi nunca son perfectamente ortogonales en la práctica ($r_1^T r_2 \neq 0$).

Si le pasamos R_{sucia} a un motor 3D, la deformación de la matriz provocará que el holograma virtual (ej. un personaje 3D) aparezca cizallado (torcido) sobre la mesa.



Para que la cámara virtual renderice sin deformaciones proyectivas, debemos encontrar la matriz de rotación pura $R_{ortogonal}$ que esté matemáticamente más cerca de R_{sucia} .

Aplicamos la Descomposición en Valores Singulares a la matriz ruidosa:

$$R_{sucia} = U\Sigma V^T$$

- ▶ Σ contiene las deformaciones de escala (cizalla).
- ▶ Para forzar que sea una rotación rígida, sustituimos Σ por la matriz Identidad I (forzamos que todos los valores singulares sean 1).

$$R_{ortogonal} = UIV^T = UV^T$$

(Opcional: asegurar que no haya reflexión verificando que el determinante sea positivo: $\det(R_{ortogonal}) \approx 1$. Si es negativo, $R_{ortogonal} = -R_{ortogonal}$).

Resultado Final: Ya podemos ensamblar la matriz de proyección final con escala métrica real, lista para multiplicar vértices 3D virtuales e incrustarlos en el vídeo:

$$M_{AR} = K[R_{ortogonal}|t]$$

Con este álgebra lineal básica, se resuelve el problema de *Pose Tracking* que permite hacer Realidad Aumentada basada en marcadores.